Decoder	Variant	$N = 48$	$N = 64$	$N = 128$	$N = 256$	$N = 512$	$N = 1024$
Linear	Autoregressive	0.01 ± 0.00	0.01 ± 0.00	0.31 ± 0.03	0.89 ± 0.03	0.97 ± 0.00	0.99 ± 0.01
	No input	0.31 ± 0.10	0.89 ± 0.05	0.98 ± 0.02	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
MLP	Autoregressive	0.02 ± 0.00	0.04 ± 0.00	0.55 ± 0.04	0.98 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
	No input	0.53 ± 0.25	0.95 ± 0.04	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
Onion	Autoregressive	0.92 ± 0.02	0.97 ± 0.01	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
	No input	0.76 ± 0.08	0.96 ± 0.01	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
GRU - autoregressive	Autoregressive	0.97 ± 0.01	0.98 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
	No input	0.92 ± 0.02	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
GRU - no input	Autoregressive	0.10 ± 0.02	0.25 ± 0.08	0.86 ± 0.01	0.99 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
	No input	0.77 ± 0.07	0.98 ± 0.01	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00

Table 4: Probe accuracy (mean of 5 runs; $\pm$ 1 s.d.).

Intervention	$N = 48$	$N = 64$	$N = 128$	$N = 256$	$N = 512$	$N = 1024$
Linear Unigram	0.06 ± 0.07	0.37 ± 0.17	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.01
Linear Bigram	0.18 ± 0.04	0.95 ± 0.06	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00	1.00 ± 0.00
Onion Unigram	0.24 ± 0.02	0.41 ± 0.04	0.76 ± 0.01	0.92 ± 0.01	0.96 ± 0.01	0.98 ± 0.00

Table 5: Intervention accuracy for GRUs without an autoregressive input in the decoding phase, with different sizes, trained on the repeat task (mean of 5 runs; $\pm$ 1 s.d.).

D GRU Models Without Autoregressive Decoding

In principle, RNN models do not need an autoregressive feedback loop during the decoding phase to be able to produce a consistent output. Given that we found that the network often relies on storing bigrams (section 6) or on onion representations (section 7), both of which benefit from autoregressive feedback, we asked what representation the models learn without such a mechanism. Thus, we changed our GRU model to receive only special PAD tokens during the decoding phase. We show the intervention accuracies in Table 5. We can see that the model is heavily based on storing unigrams, and the intervention now works down to $N = 1024$. For the $N = 64$ case, the models store bigrams. No intervention works well for the $N = 48$ non-autoregressive model, but that model also does not perform well on the validation set (see Table 3). The model is unable to to learn onion representation at any scale, since the autoregressive input is required for that, as shown in figure 4. This experiment also confirms that our subspace intervention method introduced in section 5.2 works well even for models with $N = 64$.

E Additional Discussion of the Bigram Interventions

E.1 Successful Unigram Interventions Entail Successful Bigram Interventions

With bigram interventions, in addition to copying a token to the randomized sequence, we also copy its neighborhood and replace two variables. In contrast, unigram interventions only move the corrupted token and replace its corresponding variable. Thus, the unigram intervention performs a subset of movements performed by the bigram. This means that if the unigram intervention is successful, it is guaranteed that the bigram intervention will be successful as well.

E.2 Verifying the Expressivity of the Subspace Intervention

Obtaining negative results for the unigram intervention on smaller models ($N < 512$) might raise the question of whether our intervention is expressive enough to capture the relatively small subspaces of these models. In order to verify this, we trained a GRU model without autoregressive input (Appendix D) during the decoding phase. By doing this, we eliminate some of the advantages provided by bigram representations. Since GRUs are RNNs, they can learn a decoding state machine without relying on seeing the output generated so far. We confirm this in Table 3. In these modified networks, unigram interventions are successful down to $N = 128$, and the bigram intervention is successful on all scales. We show the

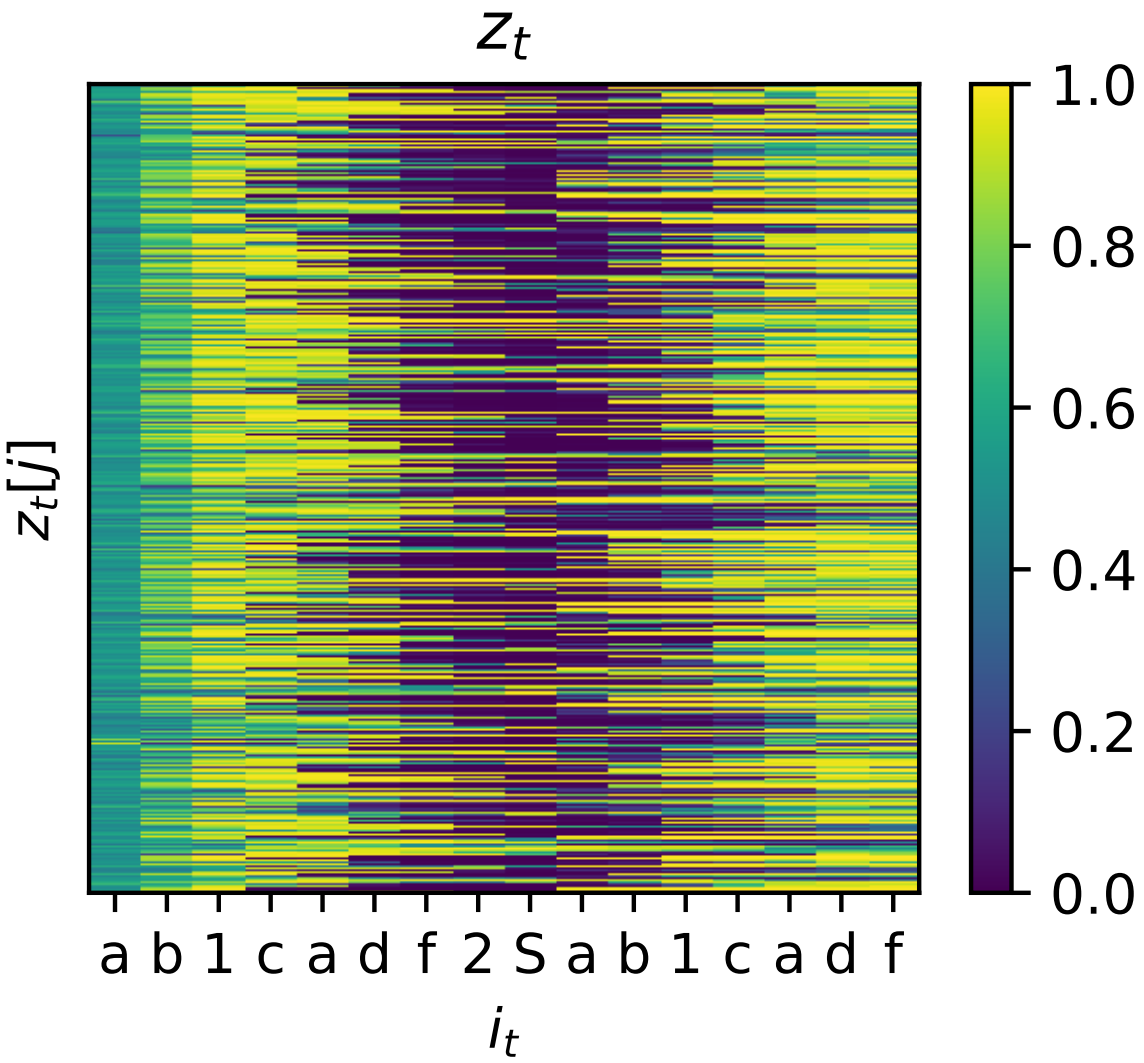

Figure 5: All 1024 channels of the GRU gate z_t shown in Figure 2a. All channels follow similar patterns.

detailed results in Table 5.

E.3 The Onion-probe

We designed a probe for onion representations similarly to the intervention described in section 7.2. We take the final representation after encoding the sequence, $\boldsymbol{h}_L$, and decode $y_L + 1 = i_1 \dots y_{2L} = i_L$ from it as follows:

$$\boldsymbol{s}_t = \boldsymbol{g}\boldsymbol{\gamma}^{t-L} + \boldsymbol{\beta}(t-L) + \boldsymbol{b} \tag{14}$$

$$y_t = \text{argmax} g(\boldsymbol{h}_{t-1}) \tag{15}$$

$$\boldsymbol{h}_t = \boldsymbol{h}_{t-1} - s_t \boldsymbol{E}[y_t] \tag{16}$$

As a denoising classifier $g(\boldsymbol{h})$ we use a 2 layer MLP with a layernorm (Ba et al., 2016) on its inputs $g(\boldsymbol{h}) = \text{softmax}\left(\boldsymbol{W}_{o_2} \max(0, \text{LN}(\boldsymbol{h}\boldsymbol{W}_{o_1} + \boldsymbol{b}_{o_1})) + \boldsymbol{b}_{o_2}\right)$, where $\text{LN}(\cdot)$ is the layernorm. Layernorm is not strictly necessary, but it greatly accelerates the learning of the probe, so we decided to keep it.

F Toy Model Implementing Onion Representations

To show more clearly how a model can learn to represent sequence elements in different scales, we constructed a toy model that uses prototypical onion representations:

$$s_t = \begin{cases} 1, & \text{if } t = 1 \\ -1, & \text{if } t = L+1 \\ \gamma s_{t-1} & \text{otherwise} \end{cases} \tag{17}$$

$$\boldsymbol{h}_1 = 0 \tag{18}$$

$$\boldsymbol{h}_{t+1} = \boldsymbol{h}_t + s_t \boldsymbol{x}_t \tag{19}$$

$$\boldsymbol{y}_t = \text{softmax}\left(\boldsymbol{h}_t \boldsymbol{W}_o + \boldsymbol{b}_o\right) \tag{20}$$

where $s_t \in \mathbb{R}$ is a scalar state representing the current scale, $\gamma \in \mathbb{R}$ represents the difference in the scales used for different variables, and $\boldsymbol{h}_t \in \mathbb{R}^N$ is the vector memory. In a real RNN, both the vector memory and the current scale are part of a single state vector. In our experiments, we use a fixed $\gamma = 0.4$. The inputs are embedded in the same way as for our GRU model: $\boldsymbol{x}_t = \boldsymbol{E}[i_t]$, where $i_t \in \mathbb{N}$ is the input token and $\boldsymbol{E} \in \mathbb{R}^{N_S \times N}$ is the embedding matrix. The only learnable parameters of this model are the embedding matrix, $\boldsymbol{E}$ and the parameters of the output projection, $\boldsymbol{W}_o \in \mathbb{R}^{N \times N}$ and $\boldsymbol{b}_o \in \mathbb{R}^N$.

The idea behind this model is based on the fact that a linear layer followed by a softmax operation is able to 'denoise' the representation $\boldsymbol{h}_t$. γ is chosen as < 0.5, because in that case the contribution to the hidden state $\boldsymbol{h}_t$ of all future $t' > t$ positions will be lower than the contribution of input $\boldsymbol{x}_t$. Thus, $\boldsymbol{x}_t$ will dominate all $\boldsymbol{h}_{t'}$ for all $t' > t$. Thus, when decoding from $\boldsymbol{h}_{t'}$, Eq. 20, followed by the argmax used in greedy decoding, the model will always recover the first, most dominant i_t that is not yet decoded from the model. Then, this token is autoregressively fed back to the next step, where it is subtracted from $\boldsymbol{h}_{t'}$, letting the next token dominate the representation $\boldsymbol{h}_{t'+1}$. This allows storing an arbitrary sequence at different scales of the representation $\boldsymbol{h}_t$. All 5 seeds of this model that we trained achieve perfect validation accuracy.